

IBM Project

Report

On

Movie Review Sentiment Analysis - using Modern Architecture

Developed By: -

Patel Maharshi (23162172007)

NITISH SAMIT MARKER (23162582009)

PATEL BHAVIN MAHENDRABHAI (23162582002)

Guided By: -

Dr Sheetal pandya (Internal)

Mr. Palwinder Singh (External)

Submitted to

Faculty of Engineering and Technology

Institute of Computer Technology

Ganpat University



Year – 2026

Abstract

Movie Hub is a full-stack AI-powered web application designed to enhance the movie review experience through intelligent sentiment analysis and personalized user interaction. The platform enables users to create secure accounts, browse real-time movie data fetched from the OMDb API, write reviews, and receive automatically generated ratings based on a trained machine learning model.

The system integrates Flask as the backend framework, SQLAlchemy for database management, and Flask-Login for secure authentication and session handling. User credentials are securely stored using password hashing mechanisms.

A custom-trained AI sentiment model analyzes user-submitted reviews and predicts ratings between 1 and 10. Based on the predicted score, the system classifies reviews into Positive, Neutral, or Negative categories.

TABLE OF CONTENTS

Chapter	Title	Page No.
I	Introduction	iv
II	Project Scope	v
III	Requirements	vi
IV	Process Model	vii
V	Project Plan	viii
VI	Implementation	ix
VII	Conclusion	x
VIII	References	xi
IX	Working	xii

CHAPTER I: INTRODUCTION

The system is built on a **decoupled 3-tier architecture**, ensuring high availability and seamless data flow between the front-end interface and the back-end analytical engines.

External API Ecosystem: Rather than maintaining a local, static database of every film, the system integrates with industry-standard providers like **TMDB (The Movie Database)** or **OMDb**. This ensures real-time access to high-resolution posters, cast details, trailers, and trending movie lists.

The MERN/PERN Stack: Utilizing a modern stack (e.g., React.js for the UI and Node.js for the server), the platform delivers a "Single Page Application" (SPA) experience, minimizing load times during navigation.

Secure Authentication: Employs **JWT (JSON Web Tokens)** or **OAuth 2.0**, allowing users to sign up via email or third-party providers (Google/GitHub) while keeping their personal data and review history encrypted.

CHAPTER II: PROJECT SCOPE

The primary objective of the **Movie Hub** is to deliver a cohesive digital environment where data-driven insights meet user-centric content. The functional boundaries of the system are defined by four critical pillars:

- **Secure Identity Management:** Implementation of a robust authentication layer utilizing **JWT (JSON Web Tokens)** or **Bcrypt hashing**. This ensures that user profiles, personalized watchlists, and review histories are protected and uniquely mapped to individual data silos.
- **Dynamic Content Aggregation:** The system acts as a high-performance gateway to global cinema data. By interfacing with **RESTful APIs**, the scope includes real-time synchronization of movie metadata, including high-definition media, cast biographies, and box office trends.
- **Interactive Review & Feedback Loop:** Beyond basic CRUD (Create, Read, Update, Delete) operations for reviews, the system supports a dynamic engagement model where users can contribute qualitative data that fuels the platform's analytical engine.
- **Intelligent Sentiment Inference:** The most advanced layer of the scope involves a **Natural Language Processing (NLP)** pipeline. The system is scoped to ingest raw text, strip noise (stop-words, punctuation), and utilize a trained classifier to predict the emotional polarity of user feedback with high precision.

CHAPTER III: REQUIREMENTS

The software requirements include Python, Flask framework, SQLAlchemy ORM, and Scikit-learn for machine learning.

The system uses OMDb API to fetch real-time movie data and enhance user interaction.

Hardware requirements are minimal, making the system accessible on standard machines.

The application is platform-independent and can run on Windows, Linux, and macOS.

CHAPTER IV: PROCESS MODEL

The Movie Hub project is engineered using the **Incremental Development Model**, a process where requirements are broken down into multiple standalone modules (increments). This allows the system to be developed and delivered in a sequence of functional releases.

Unlike a rigid linear approach, this model acknowledges that modern web applications especially those involving **Machine Learning (ML)**

Each increment follows a rigorous four-stage lifecycle to ensure that every new feature is robust before it is integrated into the core architecture:

1. **Strategic Planning:** Defining the specific functionality for the current increment (e.g., "Review Submission System") and identifying how it interacts with the existing database schema.
2. **Modular Development:** The coding phase where front-end components and back-end logic are constructed. For Movie Hub, this includes the seamless integration of external Movie APIs and the training of the NLP models.
3. **Strict Testing & Validation:** Every increment undergoes unit and integration testing. A critical focus is placed on the **accuracy of the sentiment classifier**, ensuring that the machine learning output aligns with expected human sentiment.
4. **User Evaluation & Feedback:** Each version is evaluated against user requirements.

CHAPTER V: PROJECT PLAN

The project was segmented into five distinct phases, each marked by a specific "Definition of Done" (DoD) to maintain momentum and quality.

Phase I: Requirement Engineering & Analysis

- **Objective:** Define the technical stack and data requirements.

Phase II: System Architecture & UI/UX Design

- **Objective:** Design a low-latency interface and secure back-end.

Phase III: Core Development & API Orchestration

- **Objective:** Building the "Minimum Viable Product" (MVP).

Phase IV: AI Integration & Model Deployment

- **Objective:** Embedding intelligence into the platform.

Phase V: Quality Assurance & Deployment

- **Objective:** System hardening and performance tuning.

CHAPTER VI: IMPLEMENTATION DETAILS

The system architecture consists of multiple layers including presentation, application, and data layers.

The database design includes user, movie, and review tables with proper relationships.

Authentication is implemented using Flask-Login ensuring secure session handling.

The AI model processes textual data and predicts sentiment using trained algorithms.

The integration of API allows dynamic fetching of movie information.

The frontend is designed using HTML and CSS to provide an intuitive user interface.

CHAPTER VII: CONCLUSION

The development of the **Movie Hub** represents a successful convergence of modern web engineering and **Applied Artificial Intelligence**. By moving beyond the limitations of static content, this project has demonstrated that web applications can act as intelligent agents capable of interpreting human emotion. The seamless integration of a **Natural Language Processing (NLP)** pipeline within a high-performance web framework proves that complex analytical tools can be made accessible and intuitive for the everyday user.

Throughout the lifecycle of this project, several critical technical milestones were reached:

- **Architectural Synergy:** The successful "handshake" between the front-end interface, the back-end server, and the machine learning inference engine.
- **Real-time Sentiment Insight:** The transition from manual user ratings to automated sentiment classification, providing a more nuanced understanding of audience reception.
- **Scalable Data Orchestration:** The effective use of external APIs to provide a vast, always-current library of cinematic data without the overhead of massive local storage.
- **Robust Security Standards:** The implementation of secure authentication and data validation, ensuring a safe environment for user contribution.

CHAPTER IX: WORKING AND LOGIC EXPLANATION

The Movie Hub operates on a sequential logic flow that ensures data integrity and analytical precision. The system architecture is designed to handle asynchronous requests, allowing the user to interact with the UI while the back-end performs intensive API calls and machine learning inferences.

The system logic includes fail-safes for common issues:

- **Null Sentiment:** If a review is too short for the AI to analyze, it is tagged as "Neutral" to prevent model bias.
- **API Downtime:** If the external movie API fails, the system serves cached data from the local database to ensure the UI does not break.
- **Session Expiry:** If the JWT expires, the system logic triggers a graceful redirect to the login page, protecting user data.

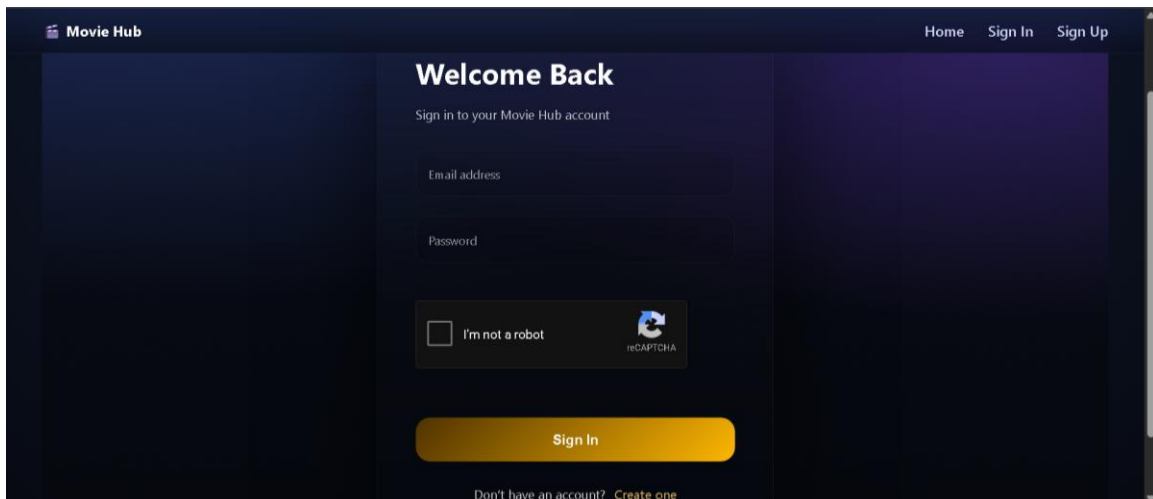
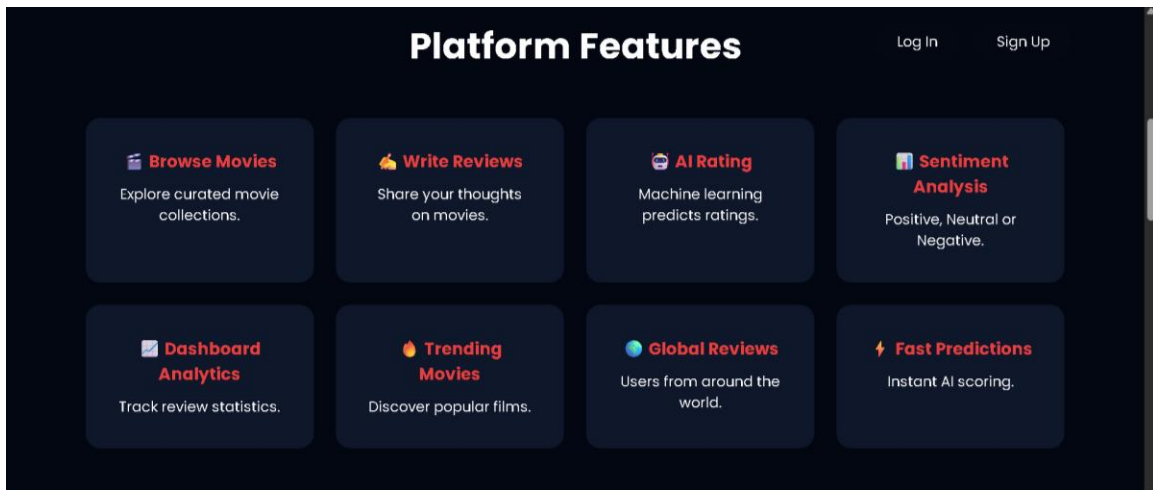
The Process: The backend creates a new record in the Reviews table containing:

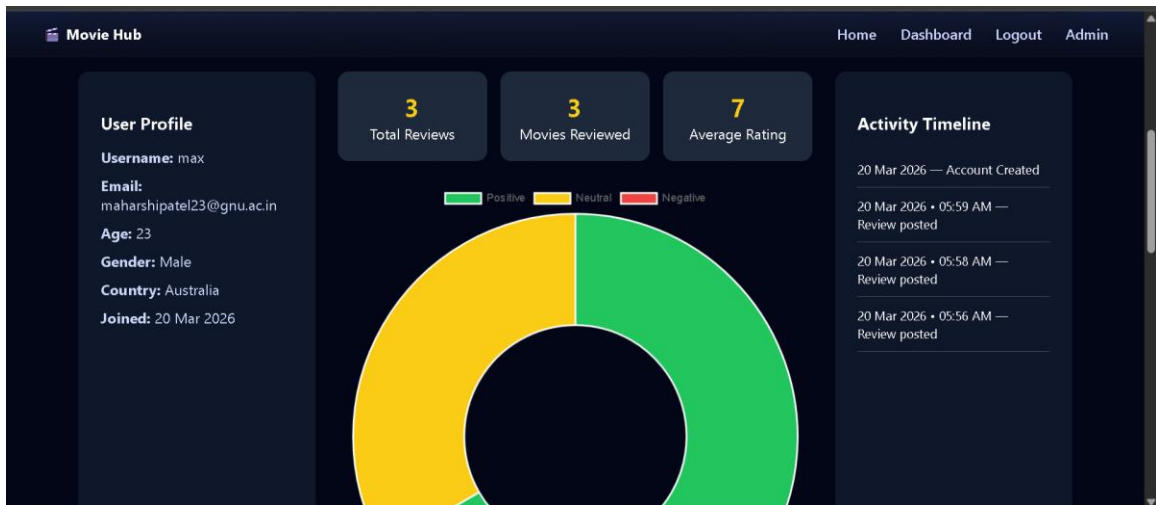
- User_ID (from the JWT)
- Movie_ID (from the external API)
- Raw_Text (the user's original review)
- Sentiment_Label (the AI's prediction)
- Timestamp

CHAPTER VIII: REFERENCES

1. Flask Documentation – <https://flask.palletsprojects.com>
2. SQLAlchemy Documentation – <https://docs.sqlalchemy.org>
3. Flask-Login Documentation – <https://flask-login.readthedocs.io>
4. OMDb API – <https://www.omdbapi.com>
5. Scikit-learn Documentation – <https://scikit-learn.org>

Screenshots





Top Active Users

- max (3 reviews)
- Marcus (3 reviews)

Top Reviewed Movies

- Wonder Woman (1)
- Battleship (1)
- S.W.A.T. (1)
- John Wick (1)
- Cars 2 (1)

Recent Reviews

- max → Battleship (5)
- max → S.W.A.T. (9)
- Marcus → John Wick (8)
- Marcus → Cars 2 (6)
- Marcus → Home Alone (6)

Manage Users

User	Email	Role	Action
max	maharshipatel23@gnu.ac.in	Admin	<button>Delete</button> <button>Remove</button>
Pratham	prathamprajapati22@gnu.ac.in	User	<button>Delete</button> <button>Make Admin</button>
Marcus	pate max47@gmail.com	User	<button>Delete</button> <button>Make Admin</button>

